Lecture Slip, CS 235
Nov 10, 2026, Tuesday
Searching Algorithms

Name:	
EmpID:	

1. **Binary search trace.** Binary search (the `lo <= hi` version from class) runs on `a = {3, 8, 12, 19, 25, 31, 44, 52, 60}` (indices 0 to 8) with target **20**. Here `mid = lo + (hi - lo) / 2`. Rows 1 and 2 are done for you. Complete the table. Use as many rows as you need.

Step	`lo`	`hi`	`mid`	`a[mid]`	**Next range**
1	0	8	4	25	$25 > 20$, so `hi = mid - 1 = 3`
2	0	3	1	8	$8 < 20$, so `lo = mid + 1 = 2`
3	2	3	2	12	$12 < 20$, so `lo = mid + 1 = 3`
4	3	3	3	19	$19 < 20$, so `lo = mid + 1 = 4`
5	4	3	–	–	`lo > hi`: stop

What does the search return? How many times was `a[mid]` compared with the target? (Do not count the `lo <= hi` test of the loop.)

Answer: The table above is the key. Four iterations run (the values of `a[mid]` are 25, 8, 12, 19). After the fourth, `lo = 4 > hi = 3`, so the loop test fails and no fifth `mid` is computed (row 5 only shows the stop). The search returns $-\mathbf{1}$ (not found), after **4** comparisons. Note that 19 and 25 were both tested: the target would have to lie between them, so it is not in the array.

2. **Growth and data structures.**

(a) In the worst case, how many comparisons does binary search need for $n = 1000$ sorted numbers? For $n = 2000$?

(b) A programmer says: "Binary search halves the range each time, so on a `std::list` (a linked list) it is also $O(\log n)$." What is wrong with that claim?

Answer: (a) The worst case is $\lfloor \log_2 n \rfloor + 1$: **10** comparisons for $n = 1000$ and **11** for $n = 2000$. Doubling the data adds **one** comparison.

(b) Halving the range is only half of the work. Binary search must also jump to the middle element, and a linked list has no random access. In a straightforward implementation, reaching the middle of the remaining range takes about half its length in pointer hops, and these add up to roughly $n/2 + n/4 + \cdots = O(n)$. Only $O(\log n)$ elements are *compared* with the target, but the pointer traversal makes the whole search $O(n)$.

3. **A one-line bug.** In the `lo <= hi` version, a student changes `hi = mid - 1;` to `hi = mid;`.

   (a) Run the changed code on `a = {5, 7}` with target **3**. Give `lo`, `hi`, `mid` for the first two iterations.

   (b) What happens next? State the rule that was broken.

   Answer: (a) Start: `lo = 0, hi = 1`. Iteration 1: `mid = 0`, `a[0] = 5 > 3`, so `hi = mid = 0`. Iteration 2: `lo = 0, hi = 0`, `mid = 0`, again `a[0] = 5 > 3`, so `hi = 0` again.

   (b) The same state repeats forever: an **infinite loop**. Rule broken: **every unsuccessful step must shrink the range**. Here `a[mid] > target`, so `mid` and everything to its right must be excluded: `hi = mid - 1`. (If `a[mid] < target`, the update is `lo = mid + 1`.) With `lo = hi = mid = 0`, the wrong assignment `hi = mid` changes nothing, so the loop never ends.

4. **Pace survey.** Please help me set the pace of the course. Circle one.

Too **slow** (I can go faster)	**About right** (normal pace)	Too **fast** (please slow down)

   Answer: Tally the three circles; there is no correct answer.

5. **Topics to clarify.** Check **all** topics you would like me to explain again next class.

   - ☐ Linear search and its cost
   - ☐ Binary search code (`lo`, `hi`, `mid`)
   - ☐ Tracing a search by hand
   - ☐ Why binary search is $O(\log n)$
   - ☐ Arrays versus linked lists for searching
   - ☐ The off-by-one bugs (`mid + 1`, `mid - 1`)
   - ☐ The code `tokenize`
   - ☐ The code `toPostfix` (shunting-yard)
   - ☐ The code `evalPostfix`
   - ☐ Other: _____

   Answer: Tally the checked topics and plan the first ten minutes of the next class around the top two.